

技術検証レポート 2026-09-30

Bend 2 を Windows なしで試す

クラウド VM での導入から、証明付き HTTP サーバー、
開発用フックまで

Bend 2.0.34 · Ubuntu 24.04 · Claude Code クラウドセッション

背景

Bend 2 とは

見た目は Python

インデント・*def*・*match* の形は Python 風。

中身は Haskell / Lean

純粋関数と IO モナド、依存型と証明、停止性の強制。

資源の扱いは Rust 寄り

変数は既定で 1 回しか使えない (affine)。複数回は *+* を付ける。

ガイドの想定は「人間が法則（仕様）を書き、AI がコードと証明を書き、コンパイラが機械的に検査する」という分担。

原文: “Bend's syntax is Python-shaped, but its semantics are closer to Haskell / Lean, while being resource-aware like Rust”

前提

Windows ネイティブ版はない

公式インストーラ

Windows では **Bend needs Linux, macOS or WSL.** を出して止まる。配布物は linux / darwin の arm64 ・ x64 のみ。

今回の検証環境

Claude Code のクラウドセッション（Ubuntu 24.04.4 / x86_64 / 4 コア / GPU なし）。clang 18.1.3 が最初から入っている。

手元の Windows に何も入れずに試せる。一方、コンテナは使い終わると回収されるので、導入し直しと成果物の保存が課題になる。

導入

公式インストーラがそのまま動いた

手順	結果
install.sh を読んでから実行	Linux x64 の SHA256 が一致、 ~/.bend に配置
hello world (bend hello.bend)	Hello, world! と表示
ネイティブビルド (-o)	clang 18 のままで成功。! 付きも CPU で動作
証明の再検査 (--verdict)	Lean 4.34.0 が必要。導入後は通過
--verdict の所要時間	初回 26.9 秒 (カーネルのビルド)、以降 0.2~0.6 秒

ガイドの要件は「clang 14+、!を使うなら 19+」。19 が必要になる条件は確かめていない。

並列実行

スレッド数に応じて速くなった

--threads	real	user
1	1.094 s	1.094 s
2	0.642 s	1.270 s
4	0.321 s	1.269 s

pow2(28n) を二分木の並列呼び出しで数える例。各条件 1 回だけの測定。

```
def pow2(+n: Nat) -> U32:
  match n:
    case 0n:
      1
    case 1n+p:
      a b = pow2(p) pow2(p)
      (a + b : U32)
```


書いてつまずいたところ

定義の順番

呼ばれる関数を上に書く。逆だと「unfilled law is a dead claim」という分かりにくいエラー。

if がない

Bool を引数に取る補助関数を作り、その中で match する。

使用回数

2 回使う変数には +。証明の中でも「consumed more than once」で数えられる。

型を書く量が多い

推論がほぼない。(a + b : U32) のように演算にも型を書く。

I/O はテキストだけではない

標準で入っている I/O

ファイル、TCP / UDP、外部コマンド実行、ウィンドウ描画、音声、並行処理（68 ファイル）。

自作の I/O

Bend 側で宣言し、中身を C と JS で書く。zlib の crc32 を呼び、Python と同じ 907060870 を得た。

落とし穴：bend の -o がリンカに渡すのは -lpthread -lm などに固定。-o *main.c* で C を出して手で clang -lz。ランタイムの *FAR* マクロが zlib と衝突する。C 側に ABI の保証はない。

155 行のサーバーを書いて動かした

パス	応答
/	200 Hello from Bend!
/hello/<name>	200 Hello, <name>!
/pow2/<d>	200 (並列計算、 $d \leq 30$)
その他	404

10 並列 × 50 リクエストはすべて 200。

見つかった不具合 1

IO.pass は失敗時にプログラム全体を終了させる。接続リセット 1 回でサーバーが落ちた → match で分けて修正。

見つかった不具合 2

String.length は文字数。日本語で Content-Length がずれた → UTF-8 のバイト数を自前で数えて修正。

仕様と証明を別ファイルに分ける

LAWS.bend（人間が書く）

「こうあるべき」を law として書く。証明のない law は未解決の主張として扱われる。

PROOF.bend（AI が書く）

同じ名前の def で証明する。bend PROOF.bend が ALL PROOFS CHECK なら完了。

```
law hello_request:
  for m: String
  for +name: String
  for r: String
  for nm: Spec.NoSpace(m)
  for nn: Spec.NoSpace(name)
  {Srv.route(Srv.path_of(m ++ (" " ++ ("/hello/" ++ name) ++ (" " ++ r))))
   == Srv.respond("200 OK", "Hello, " ++ name ++ "!\n") : String}
```

証明した法則

サーバーの純粋な部分に 6 つ

法則	内容
root_page	/ への応答は決まった挨拶
hello_echo	/hello/<name> は name をそのまま埋め込む
utf8_bytes	バイト数の関数は、仕様どおり符号化した長さに等しい
response_format	Content-Length が本文のバイト数、空行の後に本文
path_of_request	空白を含まないパスなら、リクエスト行から正しく取り出す
hello_request	上の 2 つをつなぎ、受信から応答までを 1 本で述べる

Base にはほとんど補題がなく、Nat の結合法則、`s ++ "" == s` などは自作した。`--verdict`（Lean で証明済みのカーネルによる再検査）も通過。

わざと壊して、証明が落ちるか確かめる

壊し方	結果
法則の文言を変える／実装の幅を変える／仕様の境界を変える	失敗（期待どおり）
Content-Length を文字数に戻す（M3）	最初に通ってしまった
パスの取り出し位置・区切り文字を変える	失敗（期待どおり）
/hello/ の切り取りを 6 文字にする	失敗（期待どおり）

M3 の教訓 「部品が正しい」法則だけでは「その部品を使っている」ことは保証されない。応答全体の形式を定める法則を足して塞いだ。

証明の範囲

証明しても保証されないもの

仕様そのものの誤り

法則が間違っていれば、証明が通っても不具合は残る。

I/O の部分

I/O の中身は C / JS。ガイドにも証明はホストのコードに触れないとある。

つなぎ目

handler が実際に `route(path_of(req))` を呼んでいることは述べていない。

変換処理

カーネル形式への変換には証明がない（ガイド: the translation has no proof）。

開発用フック

毎回の導入と検査を自動にした

SessionStart フック

Bend 2.0.34 を SHA256 照合つきで導入、Lean 4.34.0 を導入、カーネルを事前ビルド、PATH を設定。空の環境から 54 秒、導入済みなら 0.2 秒未満。

PostToolUse フック

.bend を編集するたびに --check-only。本体を変えたら PROOF.bend も検査し、証明が壊れたらその場で Claude に返す。

リポジトリから起動した**新しいセッション**で、両方が自動で動くことを確認した。追加の設定なしで --verdict が 0.59 秒で通過。

未解決・未確認

残っている問い

- 重い計算（/pow2/30）の最中、ほかの接続が約 0.84 秒待たされた。原因は未調査（1 回の測定）
- clang 19 が必要になる条件（今回は clang 18 で！付きも動いた）
- 新しいセッションで SessionStart フックにかかる時間（キャッシュの効き方）
- 外部ライブラリのリンクを CC のラッパーで済ませられるか
- GPU での実行（この環境には GPU がない）

まとめ

Windows なしでも、クラウド VM なら 導入から証明まで一通り試せる

動いた

導入、並列実行、外部ライブラリ、
HTTP サーバー、6 つの法則の証
明、2 つのフック。

学んだこと

証明は「書いた法則」しか守らな
い。壊して確かめる手順と、範囲外
を書き出す習慣が要る。

記録

手順・落とし穴・証明の書き方は
NOTES.md にまとめ、リポジトリで
公開。